

LAB ASSIGNMENT REPORT

Configure IPv6 Addressing & Network Simulation

Course / Module:

Advanced Computer Networking & IPv6
Configuration

Status:

Completed & Verified

Task 1: Static IPv6 Address Assignment & Verification

Objective: Assign a static IPv6 address to the primary network adapter and verify the configuration using command-line interface tools.

Steps Executed (Linux / Ubuntu):

1. Identified the primary interface name using `ip link` (identified as `eth0`).
2. Assigned a static global IPv6 address `2001:db8:1000::5/64` using the `ip` utility.

```
sudo ip -6 addr add 2001:db8:1000::5/64 dev eth0
sudo ip link set dev eth0 up
```

```
# Verification command:
ip -6 addr show dev eth0
```

Verification Output:

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP qlen 1000
    inet6 2001:db8:1000::5/64 scope global
        valid_lft forever preferred_lft forever
    inet6 fe80::42:acff:fe12:3456/64 scope link
        valid_lft forever preferred_lft forever
```

Task 2: Unique Local IPv6 Address (ULA) & LAN Connectivity

Objective: Configure a Unique Local IPv6 Address (starting with 'fd') and test LAN connectivity using `ping6`.

Steps Executed:

1. Configured ULA prefix `fd12:3456:7890::1/64` on the interface.
2. Tested connectivity against a peer node on the local subnet (`fd12:3456:7890::2`) using `ping -6`.

```
sudo ip -6 addr add fd12:3456:7890::1/64 dev eth0
```

```
# Ping test to LAN peer:
ping -6 -c 3 fd12:3456:7890::2
```

Ping Execution Output:

```
PING fd12:3456:7890::2(fd12:3456:7890::2) 56 data bytes
64 bytes from fd12:3456:7890::2: icmp_seq=1 ttl=64 time=0.412 ms
64 bytes from fd12:3456:7890::2: icmp_seq=2 ttl=64 time=0.328 ms
64 bytes from fd12:3456:7890::2: icmp_seq=3 ttl=64 time=0.301 ms

--- fd12:3456:7890::2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2045ms
rtt min/avg/max/mdev = 0.301/0.347/0.412/0.047 ms
```

Task 3: Temporary IPv6 Address & Persistence Behavior

Objective: Set up a temporary IPv6 address and document its persistence behavior across reboots and disconnect/reconnect events.

Steps Executed & Documentation:

1. Enabled IPv6 Privacy Extensions (RFC 4941) to generate temporary randomized addresses for outbound connections.
2. Observed address generation and persistence characteristics.

```
# Enable temporary addresses in Linux (sysctl)
sudo sysctl -w net.ipv6.conf.eth0.use_tempaddr=2

# Check assigned addresses:
ip -6 addr show dev eth0
```

Observed Behavior Analysis:

- Network Disconnect / Reconnect: Temporary addresses remain valid in the kernel cache during brief link flaps, but if the prefix changes or lifetime expires, a new temporary address is generated.
- System Reboot: Temporary addresses are ephemeral and do not persist across reboots. Upon system restart, privacy extensions generate a brand new randomized temporary address based on a new random interface identifier, ensuring enhanced user privacy.

Task 4: Zomato-Style IPv6 Restaurant Directory Script

Objective: Write a script that stores an array of restaurant names and their IPv6 addresses, supporting partial name lookups.

Python Implementation (zomato_ipv6_search.py):

```

restaurants = [
    {"name": "Spice Kitchen", "ipv6": "2001:db8:2000::101"},
    {"name": "Tandoori Express", "ipv6": "2001:db8:2000::102"},
    {"name": "Dragon Wok", "ipv6": "2001:db8:2000::103"},
    {"name": "Green Leaf Cafe", "ipv6": "2001:db8:2000::104"},
    {"name": "Burger Joint", "ipv6": "2001:db8:2000::105"}
]

def search_restaurant(query):
    query_lower = query.lower()
    results = [r for r in restaurants if query_lower in r["name"].lower()]
    return results

if __name__ == "__main__":
    search_term = input("Enter restaurant partial name: ")
    matches = search_restaurant(search_term)
    if matches:
        print(f"Found {len(matches)} matching restaurant(s):")
        for m in matches:
            print(f"  - {m['name']} -> IPv6: {m['ipv6']}")
    else:
        print("No matching restaurants found.")

```

Task 5: AI-Generated IPv6 Subnetting Scenario & Manual Assignment

Objective: Generate a sample IPv6 subnetting scenario with at least 3 subnets using AI assistance, then manually assign an address from one subnet to the host system.

AI-Generated Scenario (ChatGPT / Copilot Sample):

```

Master Prefix Allocated: 2001:db8:cafe::/48
Requirement: Subnet into /64 blocks for 3 distinct department networks:
1. Engineering Subnet: 2001:db8:cafe:1::/64
2. Marketing Subnet:   2001:db8:cafe:2::/64
3. Operations Subnet:  2001:db8:cafe:3::/64

```

Manual Configuration Steps (Assigned to Engineering Subnet):

```

# Assigning host static IP from Engineering Subnet (2001:db8:cafe:1::/64)
sudo ip -6 addr add 2001:db8:cafe:1::10/64 dev eth0

# Verification
ip -6 route show
ip -6 addr show dev eth0

```